

U-Net Models with Noise-Adaptive Stepping Schedules for Rectified Flow

Umna Khawaja

Samuel Zink

AI 539: Final Project Report
Oregon State University
March 17, 2026

Abstract—In generative flow models, fixed time-stepping schedules treat each discrete point as equal. This is regardless of whether or not what the model’s actual velocity estimation quality at each timestep is. This inflexibility can cause weaker velocity estimators to become a part of the generative process, degrading the sample quality. This is especially so when the number of sampling steps is small. Having been motivated by the convergence-bound framework of Chen et al. (2024), we utilize the two adaptive time-stepping algorithms: Gradient-Based Adjusting (GA) and Greedy Choosing (GC) from diffusion models and apply it to Rectified Flow (RF). To do this, we derive the Flow Matching Error (FME) as the natural RF analogue of the score matching loss. We use this to instantiate the theoretical error bound (EB). We use a lightweight U-Net Velocity model trained on MNIST and evaluated using GA-adapted, GC-adapted, and uniform schedules at $N \in \{5, 10, 20, 50\}$ sampling steps. We show that both adaptive schedules reduce EB with respect to the uniform baseline, with the most improvements in sample quality found at lower step counts. Our work shows that theory-based adaptive scheduling transfers well from the SDE setting of diffusion models to the ODE setting of Rectified Flow. All of which is done without any retraining of the underlying model. Our code is publicly available at https://github.com/umnakhawaja/AI-539-Final_Project.git.

I. INTRODUCTION

With the emergence of score-based diffusion models, generative modeling has advanced rapidly [1], [2] alongside Rectified Flow, a flow-matching framework [4]. Both types of these models generate their data by simulating a continuous-time process using discrete steps. The choice of which time points are to be used to evaluate the learned model, meaning the time-stepping schedule, is vital to the quality of the generation. However, in practice, the dominant approach is to fix the time-stepping schedule before training begins, leaving it unchanged at inference time. Choices common for this include the sub-sampled schedules of DDIM [3], the linear β -schedule constructed by Ho et al. [1], and more recently the cosine schedule of Nichol & Dhariwal [5]. Of these schedules, none adapt to the model’s learned representation—a timestep where the network produces a poor estimate of velocity is weighted the same as one where it does well.

As standardized by Chen et al. [8], this limitation is addressed by deriving an error bound (EB) which decomposes the sampling error into both a time discretization and score estimation term. From this, two algorithms are proposed: GA (Gradient-Based Adjusting), and GC (Greedy Choosing). Primarily, these minimize the EB by adjusting the discretization

points in order to “favor” areas where the model will be the most accurate. If applied to denoising diffusion models, the schedules yield quantifiable Fréchet Inception Distance (FID) improvements, shown greatly at lower step counts.

In our work, we apply GA and GC to Rectified Flow (RF). RF is a type of generative model that learns straight-line ODE paths from a Gaussian source to the data distribution. It is especially useful as a natural testbed for adaptive scheduling, as its continuous-time formulation is what allows the velocity network to be queried at any $t \in [0, 1]$. The training goal returns a clear error signal, and its Euler integrator maps directly onto the EB framework. In this paper, we train a lightweight U-Net velocity model on MNIST and compare the following three sampling schedules: Uniform (our baseline), GA, and GC, across a variety of step counts.

A. Research Questions

This paper addresses three questions:

- 1) Does the error bound EB faithfully track sample quality in Rectified Flow?
- 2) Do GA- and GC-adapted schedules outperform uniform schedules, especially at low step counts?
- 3) How does the Flow Matching Error (FME) profile shape the adapted schedules?

B. Contributions

In this paper, we make the following contributions: (i)

- 1) We service FME as the main RF analogue of score-matching loss, using it to instantiate the EB framework as done by Chen et al. [8] for Rectified Flow.
- 2) We port the GA and GC algorithms for RF with both Lipschitz and early-stopping smoothness settings, showing that both algorithms reduce EB on a model trained on MNIST.
- 3) We provide a completely open-source implementation, including training, FME measurements, schedule optimization, and evaluation guidelines.

II. BACKGROUND

A. Diffusion Models and Time-Stepping

Denoising diffusion probabilistic models (DDPMs; [1]) characterize a forward process which gradually corrupts the

data x_0 with Gaussian noise over T steps, and a reverse process that learns to denoise this. The reverse process is defined by a neural network $s_\theta(x, t)$ that is trained to approximate the score function $\nabla \ln p_t(x)$. At inference time, the reverse Stochastic Differential Equation (SDE) is produced using a discrete schedule $\{t_k\}$ of N time points. The quality of the generated data is determined by both how well s_θ calculates the true score and how the step sizes are distributed across the interval.

B. Convergence Bounds and the EB Framework

As established by Chen et al. [6], polynomial convergence is guaranteed for score-based generative models using the Girsanov’s theorem framework. Essentially, this framework decomposes the sampling error into two summed terms. The first is the *score estimation error*: the L^2 gap between the learned score s_θ and the true score $\nabla \ln p_t(x)$, as quantified by the score matching loss $D_{\text{SM}}(s_\theta, p_t)$. The second is the *time discretization error*: the approximation error computed by freezing the SDE coefficients across every interval $[t_k, t_{k+1}]$, which scales as $\mathcal{O}(h_k^2)$.

Chen et al. [8] aggregate these terms into the parameterized error bound:

$$\text{EB} = C_s \sum_{k=1}^N h_k \varepsilon(t_k) + C_L \sum_{k=1}^N h_k^2 L(t_k), \quad (1)$$

where $\varepsilon(t)$ bounds the score matching loss, $L(t)$ is a smoothness coefficient, $h_k = t_k - t_{k-1}$ is the step size, and C_s, C_L are constants independent of the schedule. Two settings are admitted for $L(t)$: the Lipschitz setting $L(t) = L^2$ (constant) [6], and the early stopping setting $L(t) = \sigma_{T-t_k}^{-4}$ [7]. Most importantly, once ε, L, C_s , and C_L are fixed, EB is a function only of the schedule $\{t_k\}$, making it a tractable optimization objective.

C. Rectified Flow

Rectified Flow (RF; [4]) is a flow-matching generative model which learns to transfer samples from a Gaussian source ($z \sim \mathcal{N}(0, I)$) to the data distribution p_0 through straight-line ODE paths. The interpolant, given a data sample x_0 and noise z , is defined as:

$$X_t = tX_1 + (1-t)X_0. \quad (2)$$

The velocity network $v_\theta(x_t, t)$ is trained to predict the constant velocity $(x_0 - z)$ via the mean squared error objective:

$$\mathcal{L}_{\text{RF}} = \mathbb{E}_{x_0, z, t} [\|v_\theta(x_t, t) - (x_0 - z)\|^2]. \quad (3)$$

At inference time, new samples are generated using Euler integration from $z \sim \mathcal{N}(0, I)$:

$$x_{t+h} = x_t + h \cdot v_\theta(x_t, t). \quad (4)$$

Rectified Flow allows for many properties that make it a good fit for adaptive scheduling. First, the velocity network is queryable at any $t \in [0, 1]$ without restriction, since time is a continuous input rather than a discrete index. This also allows

for GA, which requires the evaluation of $\varepsilon(t)$ at arbitrary points. Additionally, the training objective gives an exact error signal—the Flow Matching Error (FME)—which directly measures velocity estimation quality at any t :

$$\text{FME}(t) = \mathbb{E}[\|v_\theta(x_t, t) - (x_0 - z)\|^2]. \quad (5)$$

Lastly, RF utilizes an ODE integrator instead of an SDE, removing stochastic discretization error and making the EB framework a tighter proxy for the true sampling error.

D. Related Work on Adaptive Schedules

Many prior works address the schedule selection problem for diffusion models. DDIM [3] uses deterministic sampling by sub-selecting timesteps from a trained DDPM schedule, enabling generation with fewer steps; however, the sub-selected schedule is fixed and model-agnostic. OMS-DPM [9] trains a predictor to estimate the generation quality from a proposed schedule, but requires labeled schedule–quality pairs for training.

The approach of Chen et al. [8], and our variation of it, differs from these methods in three important respects: it is rooted in convergence theory rather than heuristic criteria, it does not require an auxiliary model or labeled quality data, and it is applicable post-hoc to any trained model without retraining. Our work extends this approach to the ODE setting of Rectified Flow, where the absence of stochastic noise allows the EB bound to be more easily interpretable.

III. METHOD

A. Adapting EB to Rectified Flow

Equation (1) was derived for the SDE setting of score-based diffusion models. We now demonstrate how it applies directly to Rectified Flow with a substitution for the error function $\varepsilon(t)$.

a) Flow Matching Error as the RF analogue of score matching loss. In diffusion models, $\varepsilon(t)$ bounds the score matching loss $D_{\text{SM}}(s_\theta, p_t) = \mathbb{E}_{x \sim p_t} [\|s_\theta(x) - \nabla \ln p_t(x)\|^2]$. However, this quantity is generally intractable in practice since the true score $\nabla \log p_t(x)$ is unknown. Prior work therefore, relies on surrogate objectives, such as denoising autoencoder (DAE) losses, as proxies when constructing the error bound. In Rectified Flow, no score function exists, but the velocity network plays the analogous role. The Flow Matching Error at time t is:

$$\text{FME}(t) = \mathbb{E}_{x_0 \sim p_0, z \sim \mathcal{N}(0, I)} [\|v_\theta(x_t, t) - (x_0 - z)\|^2]. \quad (6)$$

$\text{FME}(t)$ measures the expected squared deviation between the predicted velocity and the true constant velocity at time t . Just as D_{SM} measures how well the score network recovers the score function, FME measures how well the velocity network recovers the linear transport direction. It is the direct analogue of D_{SM} . In this work, we use $\text{FME}(t)$ as $\varepsilon(t)$ in (1), as shown here:

$$\text{EB}_{\text{RF}} = C_s \sum_{k=1}^N h_k \cdot \text{FME}(t_k) + C_L \sum_{k=1}^N h_k^2 L(t_k). \quad (7)$$

Algorithm 1 Adjusted for RF: Gradient-Based Adjusting (GA)

Input: Initial schedule $\{t_k\}_{k=0}^N$, FME function $\varepsilon(\cdot)$, smoothness $L(\cdot)$, ratio C , learning rate η

Output: Adapted schedule $(\{t_k^*\}_{k=0}^N)$

```

1: repeat
2:   for  $k = 1, \dots, N - 1$  do
3:      $g_k \leftarrow [-\varepsilon(t_{k+1}) + 2(t_k - t_{k+1}) L(t_{k+1})]$ 
4:        $+ \frac{1}{2\Delta t} [E(t_{k-1}, t_k + \Delta t) - E(t_{k-1}, t_k - \Delta t)]$ 
5:   end for
6:   for  $k = 1, \dots, N - 1$  do
7:      $t_k \leftarrow t_k - \eta g_k$ 
8:   end for
9:   Project  $\{t_k\}$  to enforce ordering and minimum
     spacing  $\gamma$ 
10: until convergence

```

b) Smoothness settings.: From Chen et al. [8], we evaluate two choices for the smoothness coefficient $L(t)$. The *Lipschitz setting* takes $L(t) = 1$ (a constant), expected when the velocity field is smooth, which is behavior expected for Rectified Flow with its straight-line paths. The *early stopping setting* takes $L(t) = \sigma_t^{-4}$ where $\sigma_t = \sqrt{t}$ represents the noise level at time t in RF. This penalizes larger steps more heavily at small t , where the interpolant is closer to the noisy source.

c) Hyperparameter C .: The ratio $C = C_L/C_S$ is a single non-negative hyperparameter balancing the two terms of EB. A larger C penalizes large step sizes much more relative to weak velocity estimates, resulting in a more evenly distributed schedule. A smaller C creates a greater concentration of steps in low-FME areas. In our experiments, we report results for $C = 1.0$ (GA) and $C = 10.0$ (GC).

B. GA: Gradient-Based Adjusting (Algorithm 1)

Gradient-Based Adjusting (GA) is applied when the velocity network can be queried at any $t \in [0, 1]$, a property Rectified Flow satisfies by construction. GA treats EB as a differentiable loss over its interior schedule points $\{t_1, \dots, t_{N-1}\}$ and minimizes it through gradient descent. A key structural observation simplifies computation: the contribution of interior point t_k to EB is confined to two adjacent terms,

$$E(t_{k-1}, t_k) + E(t_k, t_{k+1}),$$

where $E(t, s) = C_S \varepsilon(t)(s-t) + C_L L(t)(s-t)^2$. The gradient with respect to t_k is therefore local and can be computed via finite differences without evaluating the full EB sum.

The projection step ensures that adjacent points maintain a minimum separation $\gamma = 10^{-3}$ to prevent schedule collapse. The initial schedule is the uniform discretization, and $\varepsilon(\cdot)$ is estimated via Monte Carlo Sampling over a grid of points and linearly interpolated for arbitrary t . The algorithm converges when the maximum gradient magnitude falls below a threshold or after a fixed number of iterations.

Intuitively, the first term $\varepsilon(t_k) \cdot h_k$ penalizes allocating a step h_k to timesteps where FME is high, so GA redistributes steps toward regions of lower FME where the velocity network

Algorithm 2 Adjusted for RF: Greedy Choosing (GC)

Input: Pool $\{e^{(m)}\}_{m=1}^{T_1}$, initial schedule $\{t_k\}_{k=0}^N$, FME $\varepsilon(\cdot)$, smoothness $L(\cdot)$, ratio C

Output: Adapted schedule $(\{t_k^*\}_{k=0}^N)$

```

1: repeat
2:   for  $k = 1, \dots, N - 1$  do
3:      $A \leftarrow \{e^{(m)} : t_{k-1} < e^{(m)} < t_{k+1}\} \cap B(t_k, \Delta_{\max})$ 
4:      $t_k \leftarrow \arg \min_{t \in A} [E(t_{k-1}, t) + E(t, t_{k+1})]$ 
5:   end for
6: until convergence

```

is most accurate. The second term $C \cdot L(t_k) \cdot h_k^2$ penalizes individually large steps, preventing all points from collapsing into a single low-FME region.

C. GC: Greedy Choosing (Algorithm 2)

Greedy Choosing (GC) addresses the setting where the velocity network is only reliable at a finite, discrete set of time values—for instance, when time is embedded as a learned index rather than a continuous scalar. In this case, the schedule must be selected from a predefined pool of candidate points. The pool is defined as $M = 100$ uniformly spaced candidate points in $[0, 1]$. GC initializes with a uniform N -point schedule snapped to the nearest pool members, then iteratively improves it: in every inner iteration, each interior point t_k is moved to the pool candidate in the interval (t_{k-1}, t_{k+1}) that minimizes the EB contribution $E(t_{k-1}, t) + E(t, t_{k+1})$.

The set $B(t_k, \Delta_{\max})$ restricts candidates to within $\pm \Delta_{\max}$ of the current position, preventing large jumps caused by the gap between FME and the true score error at small t . GC satisfies an improvement guarantee: every inner iteration replaces t_k with a candidate that (softly) decreases EB, ensuring the converged schedule always has EB no higher than the initialization.

IV. IMPLEMENTATION

A. Model Architecture

We trained a lightweight U-Net velocity network v_θ to predict the velocity field for 28×28 grayscale MNIST images. The model architecture follows three resolution levels: $28 \times 28 \rightarrow 14 \times 14 \rightarrow 7 \times 7$, with channel widths in the encoder being $32 \rightarrow 64 \rightarrow 128$, mirrored in the decoder with skip connections. Every level has a residual block (ResBlock) succeeded by a strided convolution (encoder) or transposed convolution (decoder). At the lowest resolution, the bottleneck applies two ResBlocks. Time conditioning is applied through a sinusoidal positional embedding mapping the scalar $t \in [0, 1]$ to a 128-dimensional vector using sine and cosine features. This embedding is projected by a two-layer MLP and inserted within every ResBlock as a channel-wise bias after the primary group normalization, following Ho et al. [1]. The entire network contains approximately 1–2M parameters and can be trained in about 15 minutes on a single NVIDIA Quadro RTX 8000 GPU.

B. Training

The model is trained on the MNIST training dataset (60,000 images), all normalized to $[-1, 1]$. In every training step, we sample x_0 from the training dataset, $z \sim \mathcal{N}(0, I)$, and $t \sim \mathcal{U}[0, 1]$, compute $x_t = tX_1 + (1-t)X_0$, and minimize the MSE loss $\mathcal{L}_{\text{RF}}$ (Equation (3)). We use the AdamW optimizer [10] with learning rate 2×10^{-4} and weight decay 10^{-4} , alongside a cosine learning rate schedule over 100 epochs, batch size 128, and gradient norm clipping to 1.0.

C. FME Measurement

After training, we measure FME at $N_{\text{fme}} = 50$ uniformly distributed time points in $[0, 1]$. At every point t , $\text{FME}(t)$ is estimated using Monte Carlo Sampling using a persistent batch of 512 real images and independently sampled noise vectors. Results are linearly interpolated to form a continuous function $\varepsilon(t)$ for use in GA; for GC, ε is evaluated only at pool points $\{\hat{t}_m\}$.

A notable property of the FME profile is that it is not uniform across time. In a typical RF model, FME tends to be higher at small t (near the noisy source, where the velocity field is harder to learn) and lower at large t (near the data, where the structure is clearer). This non-uniformity is exactly what is exploited by adaptive schedules, concentrating steps in low-FME regions and allocating the computational budget where the model is most accurate.

D. Schedule Comparison Protocol

The three schedules (uniform, GA-adapted, and GC-adapted) use the same trained model with no retraining, isolating the schedule’s effect from confounding model differences. We compare schedules at four step counts, $N \in \{5, 10, 20, 50\}$. For every configuration we report: (i) EB, as computed using the measured L and ε functions; (ii) an FID proxy in pixel space, computed from the covariance and mean of the generated versus real image distributions; and (iii) qualitative sample grids for visual inspection. We evaluate both the Lipschitz ($C_L = 1$) and early stopping ($C_L(t) = \sigma_t^{-4}$) smoothness settings for GC.

V. EXPERIMENTS

In this section, we provide empirical evidence as to whether or not the GA and GC RF adaptations provide novel improvements in image generation evaluated on the MNIST dataset. For each experiment, unless stated otherwise, we used a trained model with 100 Epochs using an NVIDIA Quadro RTX 8000 GPU. We then proceed to calculate the FID and EB scores for our time-stepped scheduling algorithms (i.e. Uniform, GA, and GC) while varying various hyperparameters.

A. Initial Experiments

In the following visualizations we compare the various GA and GC algorithms against time steps $\{5, 10, 20, 50\}$ for EB and FID score with learning rate value of $5e-4$, C constant for GA to be 1, C constant for GC to be 10, and smoothing function to be Lipschitz (the same settings used in [8] for their initial experiments).

Steps	Schedule	EB	FID _{pixel}
5	Uniform	0.3742	53.688
	GA	0.3732	51.736
	GC	2.1742	51.633
10	Uniform	0.2707	46.386
	GA	0.2704	45.238
	GC	1.1712	45.530
20	Uniform	0.2178	44.723
	GA	0.2176	43.923
	GC	0.6687	44.134
50	Uniform	0.1864	43.878
	GA	0.1863	43.690
	GC	0.3674	43.990

TABLE I: Schedule data for performance with Uniform, GA and GC scheduler. Data is taken over 100+ iterations and never deviates far from these values.

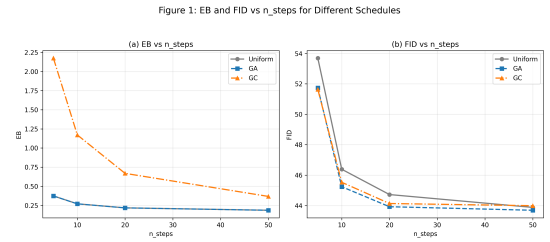
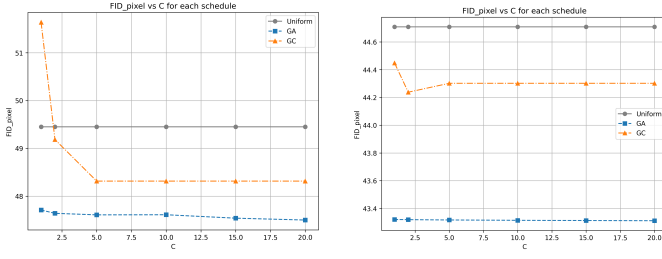


Fig. 1: Table Line Plots

In 1 we observe a large difference in the value of EB between GC and GA/Uniform. This is because we are working in a continuous time domain. When we only choose from a discrete amount of points like in GC we run the risk of not choosing scheduling points that corresponds to a local minimum in our EB. Furthermore the scalar coefficient in the calculation of the EB score for GC is set to $C=10$ indicating that it is expected for the EB value to increase. Initializing $C=10$ was initially proposed in Chen et al. (2024), so which is why we are adapting it here. However we will see that the FID is not sufficiently sensitive to our choice of C for our GC scheduling algorithm. This is likely as a result of the relative simplicity of rectified flows vs diffusion models and our limited training on a simple dataset such as MNIST. As evidence consider this plot of the FID score for our schedule algorithms with our model trained at 50 epochs as we vary our choice of C for two time partitions, 5 and 10, respectively.



(a) FID vs C for 5 steps

(b) FID vs C for 10 steps

Fig. 2: Comparison of FID vs C for different numbers of steps. From these plots we see that the hyperparameter C for our GA algorithm is largely redundant as our FID improvement is minimal. The only real sensitivity we see is for $C < 5$ for our GC algorithm. Otherwise our choice of C can be arbitrary without sufficient degradation of our FID score.

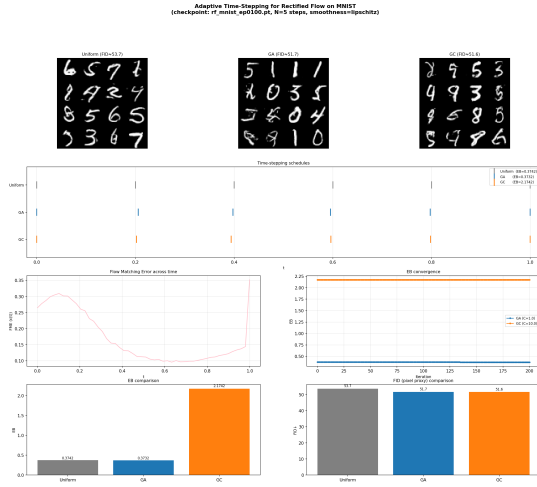


Fig. 3: Summary for 5 step schedule. In the top most section we see example generated images with different scheduling algorithms invoked. Below that with label Time-stepping schedules we see a visualization of how our GA and GC algorithms chose to repartition our scheduling points. In both cases there is a bias towards moving points to the left of our uniform baseline. Below that we plot our Flow Matching Error vs our time steps which measures how well our model predicts the velocity at said time step. To the right of that we show how our EB converge as we increase the iteration in our GA and GC algorithm. Below that are bar charts comparing the EB and FID score of each scheduling algorithm

B. Lipschitz vs Early Stopping

So far we have only considered our scheduling algorithms when the lipschitz smoothing function is used. However, since Chen 2024 [8], suggested the use of early stopping in their diffusion model version, the question is begged as to how this generalizes to RF models. Consider figure 5

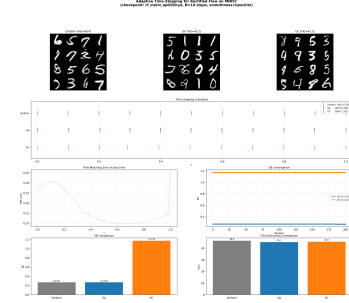


Fig. 4: Summary for 10 step schedule. Graphics are the same for 5 step schedule. We would like to show more graphics documenting experiments, but if any more are added we can no longer compile. More graphics are shown in separate doc.

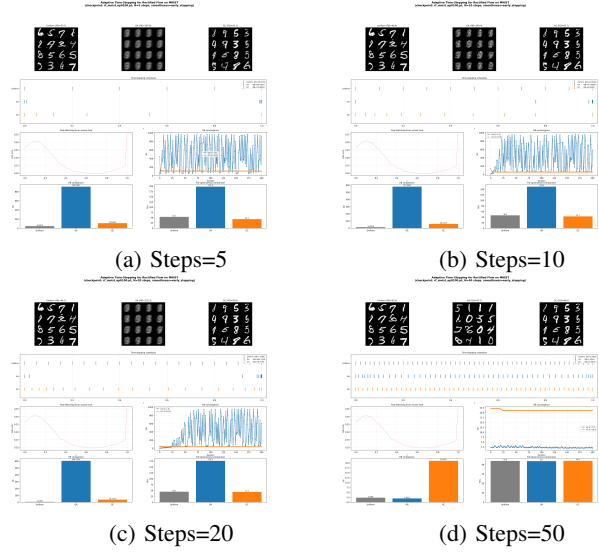


Fig. 5: Summaries of early stopping smoothing function on model trained over 100 epochs. There is no EB convergence nor images generated with relatively low FID score until 50 steps

What immediately stands out in these figures is the performance of GA. Not only is it's EB value unable to converge but it can't produce comprehensible digits as its schedule ticks are heavily biased in a small neighborhood around 1.

This might imply that early stopping was sensitive to this initialization of hyperparameters. But as we see if we vary our initialization of the C and iterations we see no increase in the FID.

In fact in these experiments 7 we observe that the uniform scheduler had an FID score of ≈ 53 where our lowest FID score for our GA scheduler was ≈ 69 . Not coming near our Uniform scheduler.

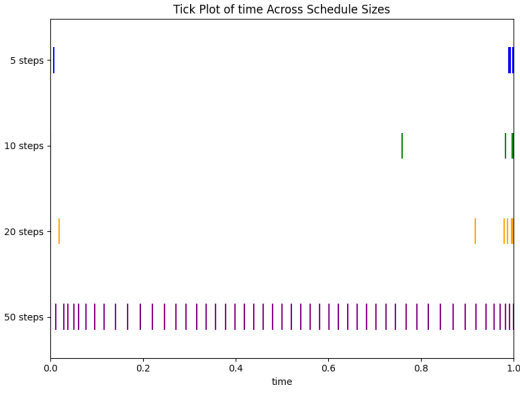


Fig. 6: Figure showing where GA chooses its new schedule points under early stopping smoothing function

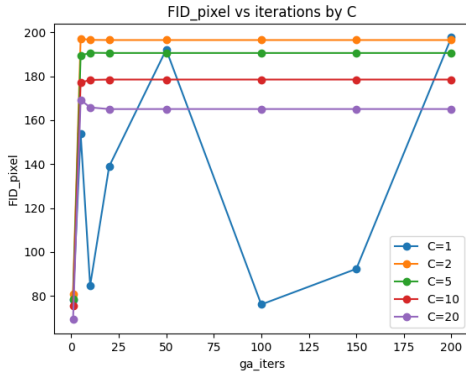


Fig. 7: Line Plot showing how hyperparameter choice effects GA scheduling algorithm with a step size of 5 and $lr=5e-4$ and a model trained for 100 Epochs.

VI. DISCUSSION

Upon analyzing the results from the experiments section we can now accurately draw conclusions about our initial research questions.

A. Response to Research Questions

We are first curious as to whether or not the error bound EB faithfully tracks the sample quality in RF. Specifically this asks how well euler integration produces quality images under our predicted velocity field and how the EB value reflects said model quality. Using the plots in 1 and 2, we see first in 1 that EB under the GC algorithm does not always reflect the sample quality. In all instances we see that GC performs better than Uniform for FID score. However, the EB value is consistently much higher than both GA and Uniform even though its sample quality performs about as well as GA and better than uniform. Furthermore, we observed that unless our C value is low, GC performs very well and does not vary when we update our EB function under Lipschitz smoothness. When we changed our smoothing function from Lipschitz to early stopping we observed that under 100 iterations it could never converge and couldn't produce novel samples.

We next asked if GA- and GC-adapted schedules outperform uniform schedules, especially at low step counts? In the initial paper Chen et al. [8] they demonstrate that these adaptive time scheduling algorithms perform particularly well in diffusion models when the step count is low when compared with overall FID score. The results we see here are analogous. When Lipschitz is used as our smoothing function we see a marginal improvement of the FID score when compared with the uniform scheduler and about a uniform FID score when we increase our step size to 20 or 50. These are similar results to what was observed in Diffusion models under Chen et al. [8].

Our final question we sought to answer was how well FME shaped the adapted schedules. Consider 2a and 2b. When comparing the FME plots and schedule plots side by side we see that our scheduling algorithms adjust the points in the direction of least FME score. This allows us to conclude that the algorithms in the RF context improve our sample quality by picking the nearest point with with lowest FME. This of course makes sense as EB is a function of FME.

B. Why does this work for Rectified Flow

We have established that we are able to achieve better image generation quality under both GA and GC. But a natural question arises as to why. Heuristically, it has been shown, [8], that the GA and GC algorithms improve performance in diffusion models, RF can be thought of as a simpler diffusion model with a linear path from noise to data (among other things), so one would expect for performance to be as good or better when compared with a uniform scheduler (like that of diffusion models). More rigorously, EB is designed to decrease over every iteration of GA and GC. Since $EB \propto FME$, if we adjust our time scheduled point such that EB decreases we are guaranteed to move to a time step where the FME value is less than or equal to the original time step or move in move closer to one of the nearest time schedules.

C. Limitations

In these experiments our biggest limitation in investigating our research questions was our dataset. MNIST is a simple dataset. Meaning that our U-Net should be able to get relatively good predictions uniformly along time, so when running our scheduling algorithm we don't move so far as we are already at an accurate time step. However, with other more complex datasets this might not be the case as when compared with other more complex datasets like CELEB-A it will be harder to create a model of this caliber. Thus, the relative simplicity of the dataset and by extension our model might explain why our GA convergence is not sensitive to our hyperparameter choice for C or our learning rate.

Another limitation is that the only metric used for evaluating generative performance was the FID. This is not sufficient for true generation quality as it has been observed that a lower FID does not always correspond with better generation.

In addition to these above limitations we were operating under a strict time limit, prohibiting certain creative directions.

D. Comparison with Original Paper

In this section we analyze the similarities and differences between our RF implementations and [8]. Both papers are trained under the MNIST dataset with [8] also using more complex datasets. Due to time constraints in our experiments we were unable to test on other datasets like Celeb A or CIFAR10 but we were able to improve in the FID score. [8] did not provide FID data on its MNIST data only visual images, however just from a visual observation it does appear that the GA and GC algorithms outperformed the fixed scheduling algorithm in [8] which is consistent with our results 3. In the FID values that were shown in [8] they showed a much larger improvement in FID scores from the fixed scheduling to adaptive scheduling for ours. There are a few possible reasons for this: 1) Their data was from Celeb A, we have not yet trained on celeb A so we do not know how our results would compare, 2) diffusion models are much more complex implying that depending on how they're trained their output could be much more sensitive to the perturbations of their time schedules.

Besides our training datasets, the obvious difference is that we are using rectified flow models while they [8] are using diffusion models. This of course makes the training easier for us but it also gives rise to some difference in theoretical performance. By this we mean the EB. In both papers $EB = C_k \sum_{k=1}^N h_k \epsilon(t_k) + h_k^2(t_k) L(t_k)$, where for [8] $\epsilon(T - t) \geq D_{SM}(s_t, p_t)$ and we use $\epsilon(T - t) = FME$. RF has the property where its analogous D_{SM} is able to be calculated. D_{SM} is only theoretical and can't be calculated with large data. This means when we run GA or GC our EB will be much more proportional to the validity of our model.

E. Future Works

We propose the following future directions for further improving on this work: 1) the most obvious improvements are in the datasets. MNIST is too simple for for constructive analysis and testing needs to be done on more complex datasets, for example Celeb A and Cifar 10, which were featured in [8]. 2) In this paper only linear RF models are studied. The natural next step is exploring RF models where the interpolation function is non-linear. 3) The only metric used for image generation quality was FID. Other methods should be explored in future works as FID is not always the best metric. Incorporating other methods such as Inception Score or Kernel Inception Score might provide more insight. As another way to measure the image quality one could also train a discriminator or a classifier as a way to infer the quality of our images. 4) In diffusion models this initial groundwork was based around the idea that D_{SM} could not be computed. In the RF analogy we are able to compute the FME implying that some methods, for example the early stopping smoothness function, could be better finetuned for RF use.

VII. CONCLUSION

We considered the problem of generalizing Chen's et al [8] algorithms for selecting discretizations points to RF models.

We've showed that when adapted to RF models we achieve a marginal improvement over MNIST datasets. GA and GC improve generating performance for RF models among all step sizes but especially with fewer discretizations points.

We hypothesize that our methods for schedule point adjustments can be improved further by specializing GA and GC specifically for RF. We further conjecture that these methods will show increased performance as we increase the complexity of our dataset, use worse models, and constrain our sampling steps to small values.

REFERENCES

- [1] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [2] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, "Score-based generative modeling through stochastic differential equations," in *International Conference on Learning Representations (ICLR)*, 2021.
- [3] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," in *International Conference on Learning Representations (ICLR)*, 2021.
- [4] X. Liu, C. Gong, and Q. Liu, "Flow straight and fast: Learning to generate and transfer data with rectified flow," in *International Conference on Learning Representations (ICLR)*, 2023.
- [5] A. Q. Nichol and P. Dhariwal, "Improved denoising diffusion probabilistic models," in *International Conference on Machine Learning (ICML)*, 2021.
- [6] S. Chen, S. Chewi, J. Li, Y. Li, A. Salim, and A. Zhang, "Sampling is as easy as learning the score: theory for diffusion models with minimal data assumptions," in *International Conference on Learning Representations (ICLR)*, 2023.
- [7] S. Chen, G. Meng, and A. Zhang, "The probability flow ODE is provably fast," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [8] S. Chen et al., "Adaptive time-stepping schedules for score-based generative models," 2024.
- [9] L. Liu et al., "OMS-DPM: Optimizing the model schedule for diffusion probabilistic models," in *International Conference on Machine Learning (ICML)*, 2023.
- [10] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations (ICLR)*, 2019.